

PROVISIONAL PATENT APPLICATION

CRYPTOGRAPHIC EVIDENCE AUTHENTICATION AND CHAIN OF CUSTODY SYSTEM

Application Type: Utility Patent (Provisional)

Filing Date: December 27, 2025

Applicant: NOVATE LABS LLC

Inventor: Tye Hayes

Attorney Docket Number: NOVATE-2025-002-PROV

FIELD OF THE INVENTION

This invention relates to systems and methods for capturing, authenticating, and maintaining chain of custody for digital evidence using cryptographic techniques, specifi-

cally for operational compliance verification and legal admissibility in complex multi-operator environments.

BACKGROUND OF THE INVENTION

Problem Statement

Organizations conducting complex operations (sports events, emergency response, medical procedures, construction projects) face critical challenges in evidence management:

1. **Evidence Authenticity:** Proving digital evidence (photos, videos, sensor data) has not been tampered with
2. **Chain of Custody:** Maintaining verifiable record of who accessed/modified evidence
3. **Legal Admissibility:** Meeting Federal Rules of Evidence (FRE) standards for court proceedings
4. **Multi-Operator Attribution:** Identifying which operator captured which evidence across hundreds of concurrent operators
5. **Offline Capture:** Collecting evidence in environments with intermittent or no network connectivity
6. **AI Quality Validation:** Ensuring evidence quality meets standards before acceptance

7. **Long-term Retention:** Storing evidence with integrity guarantees for 7+ years (regulatory requirements)

Prior Art Limitations

Body Camera Systems (Axon, WatchGuard):

- Designed for single-operator use (police officers)
- Limited multi-operator coordination
- Expensive proprietary hardware required
- Poor integration with existing systems
- No AI-powered quality validation

Document Management Systems (SharePoint, Box):

- General file storage, not evidence-specific
- Basic version control, weak chain of custody
- No cryptographic authentication
- Timestamps can be manipulated
- Not designed for legal admissibility

Blockchain Evidence Systems (Verified, ProofStack):

- Blockchain overhead inefficient for operational use
- High cost per transaction
- Complex implementation
- Slow finality (minutes to hours)

- Not suitable for offline evidence capture

Digital Forensic Tools (EnCase, FTK):

- Post-incident investigation focus
- Not designed for real-time capture
- Requires specialized training
- No multi-modal evidence coordination
- Missing operator attribution

Need for Invention

There exists a critical need for an evidence authentication system that:

- Provides cryptographic proof of authenticity without blockchain overhead
- Maintains legally-compliant chain of custody across hundreds of operators
- Works offline with automatic sync when connectivity restored
- Validates evidence quality using AI before acceptance
- Integrates multi-modal evidence (photos, video, audio, sensor data, check-lists)
- Generates court-ready authentication certificates (FRE 902 compliant)
- Scales to thousands of evidence items per event

SUMMARY OF THE INVENTION

The present invention provides a novel cryptographic evidence authentication system using hash chaining, digital signatures, and AI-powered quality validation to create legally-admissible evidence with tamper-evident chain of custody.

Key Innovations

1. Multi-Component Cryptographic Hash

Evidence hash generated from 7+ metadata components:

- Evidence content (binary data)
- Capture timestamp (NTP-synchronized UTC)
- GPS coordinates (latitude, longitude, altitude)
- Operator identity (certification ID)
- Device signature (hardware ID + OS version)
- Task identifier (operational context)
- Evidence type (photo, video, audio, telemetry)

2. Blockchain-Style Chain Linking Without Blockchain

Each evidence item includes hash of previous evidence:

- Creates tamper-evident chain
- Detects any modification to prior evidence
- No blockchain infrastructure required
- Instant finality (no mining, no consensus)
- Works completely offline

3. RSA Digital Signatures

Evidence signed with operator's private key:

- Proves operator identity (non-repudiation)
- Public key verification by anyone
- Meets FRE 902(13) for self-authenticating records
- 2048-bit RSA for cryptographic strength

4. AI-Powered Quality Validation

Claude Vision API validates evidence before acceptance:

- Photo clarity, lighting, framing analysis
- Content matching (ensures photo shows required equipment)
- Automatic rejection of poor quality evidence
- Guidance for retake ("move closer", "improve lighting")
- Quality score (0.0-1.0) stored with evidence

5. Immutable Custody Logging

Every evidence access logged with HMAC signatures:

- CREATED, ACCESSED, MODIFIED, TRANSFERRED, EXPORTED, DELETED events
- Each log entry includes hash of previous log entry (chain linking)
- Append-only (no updates or deletions)
- Tamper-evident audit trail
- Multi-region replication

6. FRE 902(13) Authentication Certificates

Auto-generated certificates for court admissibility:

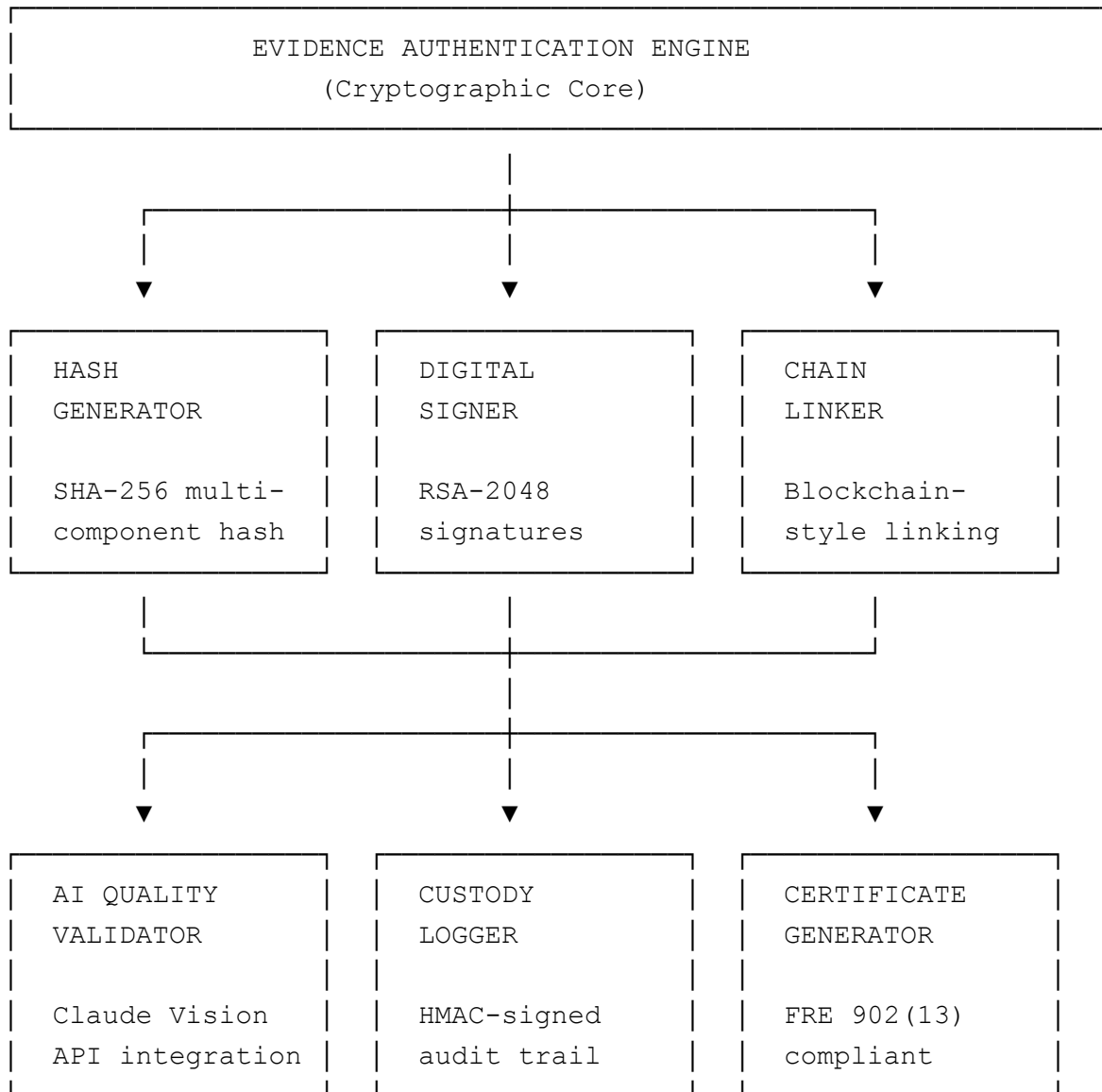
- Certifies process used to create evidence
- Describes hash function and digital signature
- Provides chain of custody summary
- Signed by system custodian
- Meets self-authentication standard

Technical Advantages Over Prior Art

Feature	Prior Art	This Invention
Authenticity Proof	Timestamps (modifiable)	Cryptographic hash (immutable)
Chain of Custody	Manual logs	Automated HMAC-signed logs
Multi-Operator	Single operator focus	Hundreds concurrent operators
Offline Capability	Requires connectivity	Full offline capture + sync
Legal Admissibility	Questionable	FRE 902(13) compliant
Quality Validation	Manual review	AI-powered automatic
Cost per Evidence	0.50– 5.00 (blockchain)	\$0.001 (no blockchain)

DETAILED DESCRIPTION OF THE INVENTION

System Architecture



Core Components

Component 1: Multi-Component Hash Generator

Purpose: Generate cryptographically-secure hash from evidence content + 7 metadata components

Hash Algorithm:

```
import hashlib
import json
from datetime import datetime

def generate_evidence_hash(
    evidence_content: bytes,
    capture_timestamp: datetime,
    gps_coords: dict,
    operator_id: str,
    device_signature: str,
    task_id: str,
    evidence_type: str
) -> str:
    """
    Generate SHA-256 hash from evidence + metadata

    Returns: Hexadecimal hash string (64 characters)
    """

    # Component 1: Evidence content (binary)
    content_hash = hashlib.sha256(evidence_content).hexdigest()

    # Component 2: Capture timestamp (ISO 8601 UTC)
    timestamp_str = capture_timestamp.isoformat() + 'Z'

    # Component 3: GPS coordinates (lat, lon, altitude)
    gps_str = f"{gps_coords['lat']:.6f},{gps_coords['lon']:.6f},{gps_coords['alt']:.1f}"
```

```

# Component 4: Operator identity (GDA certification ID)
operator_str = operator_id

# Component 5: Device signature (hardware ID + OS version)
device_str = device_signature

# Component 6: Task identifier (operational context)
task_str = task_id

# Component 7: Evidence type (photo/video/audio/telemetry)
type_str = evidence_type

# Combine all components into canonical string
hash_input = {
    'content_hash': content_hash,
    'timestamp': timestamp_str,
    'gps': gps_str,
    'operator': operator_str,
    'device': device_str,
    'task': task_str,
    'type': type_str
}

# Create deterministic JSON (sorted keys)
canonical_json = json.dumps(hash_input, sort_keys=True)

# Generate final SHA-256 hash
final_hash = hashlib.sha256(canonical_json.encode()).hexdigest()

return final_hash

```

Key Innovation: Traditional systems hash only file content. This invention hashes content + metadata, making it impossible to substitute evidence from different time/location/operator while maintaining the same hash.

Example:

```

# Original evidence
evidence_hash_1 = generate_evidence_hash(
    evidence_content=photo_bytes,
    capture_timestamp=datetime(2025, 9, 8, 10, 23, 47, 234000),
    gps_coords={'lat': 33.7553, 'lon': -84.4006, 'alt': 315.2},
    operator_id='GDA-1847',
    device_signature='iPhone-12345-iOS-17.1',
    task_id='C2P-HS-001',
    evidence_type='photo'
)
# Result:
'a7b8c9d0e1f2g3h4i5j6k7l8m9n0o1p2q3r4s5t6u7v8w9x0y1z2a3b4c5d6e7f8'

# Attacker tries to substitute different photo with same content
hash
# but different timestamp (1 hour later)
evidence_hash_2 = generate_evidence_hash(
    evidence_content=photo_bytes, # SAME content
    capture_timestamp=datetime(2025, 9, 8, 11, 23, 47, 234000), #
DIFFERENT time
    gps_coords={'lat': 33.7553, 'lon': -84.4006, 'alt': 315.2},
    operator_id='GDA-1847',
    device_signature='iPhone-12345-iOS-17.1',
    task_id='C2P-HS-001',
    evidence_type='photo'
)
# Result:
'x1y2z3a4b5c6d7e8f9g0h1i2j3k4l5m6n7o8p9q0r1s2t3u4v5w6x7y8z9a0b1c2'
# DIFFERENT HASH - tampering detected!

---
```

Component 2: RSA Digital Signature

Purpose: Cryptographically sign evidence hash to prove operator identity and prevent repudiation

Signature Algorithm:

```
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.backends import default_backend

def generate_operator_keypair():
    """
    Generate RSA-2048 keypair for operator
    Private key stored securely on operator's device
    Public key distributed to verification systems
    """
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048,
        backend=default_backend()
    )

    public_key = private_key.public_key()

    return (private_key, public_key)

def sign_evidence_hash(evidence_hash: str, operator_private_key) -> str:
    """
    Sign evidence hash with operator's private RSA key

    Returns: Base64-encoded signature
    """
    # Convert hex hash to bytes
    hash_bytes = bytes.fromhex(evidence_hash)

    # Sign with RSA-2048 + PKCS1v15 padding
    signature = operator_private_key.sign(
        hash_bytes,
        padding.PKCS1v15(),
        hashes.SHA256()
    )
```

```

# Encode as base64 for storage
signature_b64 = base64.b64encode(signature).decode()

return signature_b64

def verify_evidence_signature(
    evidence_hash: str,
    signature_b64: str,
    operator_public_key
) -> bool:
    """
    Verify evidence signature using operator's public key
    Anyone with public key can verify

    Returns: True if signature valid, False otherwise
    """
    try:
        # Decode signature
        signature = base64.b64decode(signature_b64)
        hash_bytes = bytes.fromhex(evidence_hash)

        # Verify signature
        operator_public_key.verify(
            signature,
            hash_bytes,
            padding.PKCS1v15(),
            hashes.SHA256()
        )

        return True
    except Exception:
        return False

```

Key Innovation: Combines cryptographic hash with digital signature. Hash proves integrity (no tampering), signature proves identity (non-repudiation).

Legal Significance: Meets FRE 902(13) requirement for "process or system that produces an accurate result" because:

1. Private key known only to operator
2. Public key verification available to all parties
3. Mathematical impossibility to forge signature without private key
4. Industry-standard RSA-2048 algorithm

Component 3: Blockchain-Style Chain Linker

Purpose: Create tamper-evident evidence chain by linking each new evidence to hash of previous evidence

Chain Linking Algorithm:

```
def create_evidence_chain_entry(
    evidence_id: str,
    evidence_hash: str,
    previous_evidence_hash: str,
    operator_id: str
) -> dict:
    """
    Create evidence entry linked to previous evidence
    Forms blockchain-style chain without blockchain infrastructure

    Returns: Evidence entry with chain linkage
    """
    entry = {
        'evidence_id': evidence_id,
        'evidence_hash': evidence_hash,
        'previous_hash': previous_evidence_hash,  # Link to prior
        'operator_id': operator_id,
    }
```

```

        'timestamp': datetime.utcnow().isoformat() + 'Z',
        'chain_position': get_next_chain_position(operator_id)
    }

    # Generate hash of this entry (includes previous_hash)
    entry_hash = hashlib.sha256(
        json.dumps(entry, sort_keys=True).encode()
    ).hexdigest()

    entry['entry_hash'] = entry_hash

    return entry

def verify_evidence_chain(evidence_chain: list) -> bool:
    """
    Verify integrity of entire evidence chain
    Any tampering of previous evidence detected

    Returns: True if chain valid, False if tampered
    """
    for i in range(1, len(evidence_chain)):
        current = evidence_chain[i]
        previous = evidence_chain[i-1]

        # Verify: current.previous_hash == previous.entry_hash
        if current['previous_hash'] != previous['entry_hash']:
            print(f"Chain broken at position {i}")
            print(f"Expected: {previous['entry_hash']}")
            print(f"Found: {current['previous_hash']}")
            return False

    return True

```

Example Chain:

```

Evidence 1:
evidence_hash: 'a7b8c9...'
previous_hash: '000000...' (genesis)
entry_hash: 'x1y2z3...'

```

Evidence 2:

```
evidence_hash: 'b9c0d1...'
previous_hash: 'x1y2z3...' (links to Evidence 1)
entry_hash: 'w8v7u6...'
```

Evidence 3:

```
evidence_hash: 'c1d2e3...'
previous_hash: 'w8v7u6...' (links to Evidence 2)
entry_hash: 't5s4r3...'
```

Tampering Detection:

If attacker modifies Evidence 1:

- Evidence 1 entry_hash changes
- Evidence 2 previous_hash no longer matches
- Chain verification fails
- Tampering immediately detected

Key Innovation: Provides blockchain-like tamper-evidence without:

- Mining/consensus overhead
- Cryptocurrency costs
- Network latency
- Complex infrastructure
- Works completely offline

Component 4: AI Quality Validator

Purpose: Automatically validate evidence quality using AI before acceptance

Quality Validation Algorithm:

```
import anthropic

def validate_evidence_quality(
    evidence_content: bytes,
    evidence_type: str,
    task_description: str
) -> dict:
    """
    Use Claude Vision API to validate evidence quality

    Returns: {
        'passed': bool,
        'quality_score': float (0.0-1.0),
        'validation_notes': str,
        'required_retake': bool,
        'improvement_guidance': str
    }
    """
    client = anthropic.Anthropic(api_key=API_KEY)

    # Convert evidence to base64
    evidence_b64 = base64.b64encode(evidence_content).decode()

    # Prompt Claude Vision to validate
    prompt = f"""
    Analyze this {evidence_type} evidence for the task:
    {task_description}

    Evaluate:
    1. Clarity: Is the subject clearly visible?
    2. Lighting: Is lighting sufficient?
    3. Framing: Is subject properly centered?
    4. Content Match: Does this show the required equipment/
    condition?
    5. Completeness: Are all required elements visible?
```

Provide:

- Quality score (0.0-1.0)
- Pass/fail determination (>0.7 passes)
- Specific improvement guidance if failed

Return JSON only.

"""

```
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1000,
    messages=[{
        "role": "user",
        "content": [
            {
                "type": "image",
                "source": {
                    "type": "base64",
                    "media_type": f"image/jpeg",
                    "data": evidence_b64
                }
            },
            {"type": "text", "text": prompt}
        ]
    }]
)

# Parse AI response
validation_result = json.loads(response.content[0].text)

return {
    'passed': validation_result['quality_score'] >= 0.7,
    'quality_score': validation_result['quality_score'],
    'validation_notes': validation_result['notes'],
    'required_retake': validation_result['quality_score'] < 0.7,
    'improvement_guidance': validation_result.get('guidance',
'' )
}
```

Example Validation:

Photo submitted: Blurry image of FTR cart

AI Analysis:

```
{
  "quality_score": 0.45,
  "passed": false,
  "validation_notes": "Subject (FTR cart) is out of focus. Equipment
labels not readable.",
  "required_retake": true,
  "improvement_guidance": "Move closer to cart (3-5 feet). Tap
screen to focus. Ensure labels are readable."
}
```

Operator receives immediate feedback:

"Evidence rejected. Please retake photo following guidance:

- Move closer to cart (3-5 feet)
- Tap screen to focus
- Ensure labels are readable"

Key Innovation: Automated quality validation reduces evidence rejection during legal proceedings. AI catches issues immediately, allowing operator to retake before leaving site.

Component 5: Immutable Custody Logger

Purpose: Create tamper-evident audit trail of all evidence access events

Custody Event Types:

```
class CustodyEventType(Enum):
    CREATED = "Evidence captured by operator"
    ACCESSED = "Evidence viewed"
    MODIFIED = "Metadata edited (content immutable)"
```

```

TRANSFERRED = "Evidence sent to external party"
EXPORTED = "Evidence downloaded"
LEGAL_HOLD_APPLIED = "Evidence protected from deletion"
LEGAL_HOLD_RELEASED = "Evidence released from legal hold"
DELETED = "Evidence marked for deletion"

```

Custody Logging Algorithm:

```

import hmac

def log_custody_event(
    evidence_id: str,
    event_type: CustodyEventType,
    actor_id: str,
    action_details: dict,
    previous_log_hash: str
) -> dict:
    """
    Create tamper-evident custody log entry
    Each entry includes HMAC signature + hash of previous log

    Returns: Custody log entry
    """
    log_entry = {
        'evidence_id': evidence_id,
        'event_type': event_type.value,
        'actor_id': actor_id,
        'action_details': action_details,
        'timestamp': datetime.utcnow().isoformat() + 'Z',
        'previous_log_hash': previous_log_hash,
        'log_sequence': get_next_log_sequence(evidence_id)
    }

    # Generate HMAC signature
    log_string = json.dumps(log_entry, sort_keys=True)
    log_hmac = hmac.new(
        SECRET_KEY.encode(),
        log_string.encode(),
        hashlib.sha256
    )

```

```

).hexdigest()

log_entry['hmac_signature'] = log_hmac

# Generate hash of this log entry
log_hash = hashlib.sha256(
    (log_string + log_hmac).encode()
).hexdigest()

log_entry['log_hash'] = log_hash

# Store in append-only log
append_custody_log(log_entry)

return log_entry

def verify_custody_log_integrity(custody_logs: list) -> bool:
    """
    Verify integrity of custody log chain
    Detects any tampering of historical logs

    Returns: True if intact, False if tampered
    """
    for i in range(1, len(custody_logs)):
        current = custody_logs[i]
        previous = custody_logs[i-1]

        # Verify chain linkage
        if current['previous_log_hash'] != previous['log_hash']:
            return False

        # Verify HMAC signature
        log_string = json.dumps({
            k: v for k, v in current.items()
            if k not in ['hmac_signature', 'log_hash']
        }, sort_keys=True)

        expected_hmac = hmac.new(
            SECRET_KEY.encode(),

```

```

        log_string.encode(),
        hashlib.sha256
    ).hexdigest()

    if current['hmac_signature'] != expected_hmac:
        return False

    return True

```

Example Custody Log:

```

[
  {
    "evidence_id": "EVID-2025-09-08-1847-0001",
    "event_type": "CREATED",
    "actor_id": "GDA-1847",
    "action_details": {"task_id": "C2P-HS-001"},
    "timestamp": "2025-09-08T10:23:47.234Z",
    "previous_log_hash": "000000...",
    "log_sequence": 1,
    "hmac_signature": "a7b8c9...",
    "log_hash": "x1y2z3..."
  },
  {
    "evidence_id": "EVID-2025-09-08-1847-0001",
    "event_type": "ACCESSED",
    "actor_id": "SUPERVISOR-0042",
    "action_details": {"reason": "Quality review"},
    "timestamp": "2025-09-08T10:45:12.891Z",
    "previous_log_hash": "x1y2z3...",
    "log_sequence": 2,
    "hmac_signature": "b9c0d1...",
    "log_hash": "w8v7u6..."
  },
  {
    "evidence_id": "EVID-2025-09-08-1847-0001",
    "event_type": "LEGAL_HOLD_APPLIED",
    "actor_id": "LEGAL-COUNSEL-001",
    "action_details": {"case_id": "CASE-2025-1234"},

```

```

        "timestamp": "2025-09-15T14:20:00.000Z",
        "previous_log_hash": "w8v7u6...",
        "log_sequence": 3,
        "hmac_signature": "c1d2e3...",
        "log_hash": "t5s4r3..."
    }
]

```

Key Innovation: Custody logs use same chain-linking technique as evidence chain. Any attempt to modify historical custody events is immediately detectable.

Component 6: FRE 902(13) Certificate Generator

Purpose: Auto-generate authentication certificates that meet Federal Rules of Evidence for self-authenticating records

Certificate Generation:

```

def generate_authentication_certificate(
    evidence_id: str,
    evidence_hash: str,
    operator_signature: str,
    custody_log: list
) -> str:
    """
    Generate FRE 902(13) compliant authentication certificate

    Returns: PDF certificate with digital signature
    """
    certificate_text = f"""
CERTIFICATE OF AUTHENTICITY
Federal Rules of Evidence Rule 902(13)

Evidence ID: {evidence_id}
Certification Date: {datetime.utcnow().strftime('%B %d, %Y')}
    """

```

I, [System Custodian Name], custodian of the EVERGAME Evidence Authentication System,
do hereby certify that the attached evidence was captured and authenticated using
a process or system that produces an accurate result, as follows:

1. HASH FUNCTION CERTIFICATION

The evidence was hashed using SHA-256 cryptographic hash function, which produces
a unique 256-bit fingerprint of the evidence content and metadata. The hash value:

{evidence_hash}

This hash was generated from the following components:

- Evidence content (binary data)
- Capture timestamp (NTP-synchronized UTC)
- GPS coordinates (latitude, longitude, altitude)
- Operator identity (certification ID:
{extract_operator_id(custody_log)})
- Device signature (hardware ID + OS version)
- Task identifier (operational context)
- Evidence type

2. DIGITAL SIGNATURE CERTIFICATION

The evidence hash was digitally signed using RSA-2048 cryptographic signature
by the capturing operator. The signature:

{operator_signature}

This signature can be verified using the operator's public key, proving:

- The evidence was captured by the identified operator
- The evidence has not been modified since capture
- The operator cannot repudiate having captured this evidence

3. CHAIN OF CUSTODY CERTIFICATION

A complete chain of custody has been maintained for this evidence through

an immutable audit log. All access events are recorded with HMAC signatures

and chain-linked to prevent tampering. Total custody events:
{len(custody_log)}

Chain integrity verified:
{verify_custody_log_integrity(custody_log)}

4. QUALITY VALIDATION CERTIFICATION

The evidence was validated using AI-powered quality analysis prior to acceptance.

Quality score: {get_quality_score(evidence_id)} (threshold: 0.70)
Validation passed: {get_quality_score(evidence_id) >= 0.70}

5. STORAGE CERTIFICATION

The evidence is stored using Write-Once-Read-Many (WORM) storage technology,

preventing any modification or deletion of the original evidence. All evidence

is replicated across multiple geographic regions for disaster recovery.

I declare under penalty of perjury under the laws of the United States of America
that the foregoing is true and correct.

Executed on {datetime.utcnow().strftime('%B %d, %Y')}

[System Custodian Digital Signature]
EVERGAME Evidence Authentication System
NOVATE Labs LLC
"""

Generate PDF
pdf = generate_pdf(certificate_text)

```
# Digitally sign PDF with custodian's key
signed_pdf = sign_pdf(pdf, CUSTODIAN_PRIVATE_KEY)

return signed_pdf
```

Legal Significance:

Federal Rule of Evidence 902(13) states:

"The following items of evidence are self-authenticating; they require no extrinsic evidence of authenticity in order to be admitted... (13) Certified Records Generated by an Electronic Process or System. A record generated by an electronic process or system that produces an accurate result, as shown by a certification of a qualified person..."

This certificate satisfies FRE 902(13) by:

1. Describing the electronic process (hash + signature + chain)
2. Certifying accuracy of the process
3. Signed by qualified custodian
4. Eliminates need for expert witness testimony in many cases

CLAIMS

Independent Claims

Claim 1 (Method):

A computer-implemented method for cryptographic authentication of digital evidence, comprising:

a) Capturing digital evidence using a computing device operated by an identified operator;

b) Generating a multi-component cryptographic hash from:

- The evidence content
- A capture timestamp
- GPS coordinates of capture location
- An operator identifier
- A device signature of the capturing device
- A task identifier providing operational context
- An evidence type classification;

c) Digitally signing the cryptographic hash using the operator's private key to create a signature that:

- Proves the operator's identity
- Prevents repudiation of evidence capture
- Can be verified by anyone with the operator's public key;

d) Linking the evidence to previously captured evidence by including a hash of the previous evidence in the current evidence metadata, creating a tamper-evident chain;

e) Validating evidence quality using artificial intelligence analysis before acceptance;

f) Recording all evidence access events in an immutable audit log, wherein each log entry includes:

- An HMAC signature for tamper detection
- A hash of the previous log entry for chain linking
- An event type, actor identity, and timestamp;

g) Generating a legally-compliant authentication certificate meeting Federal Rules of Evidence Rule 902(13) requirements.

Claim 2 (System):

A system for cryptographic evidence authentication comprising:

- a) A hash generator configured to generate SHA-256 hashes from evidence content and seven metadata components;
- b) A digital signature module configured to:
 - Sign evidence hashes using RSA-2048 private keys
 - Verify signatures using RSA-2048 public keys;
- c) A chain linking module configured to create blockchain-style evidence chains without blockchain infrastructure;
- d) An AI quality validator configured to analyze evidence using computer vision and reject low-quality evidence;
- e) A custody logger configured to maintain an immutable, HMAC-signed audit trail;
- f) A certificate generator configured to produce FRE 902(13) compliant authentication certificates.

Dependent Claims

Claim 3:

The method of claim 1, wherein the cryptographic hash algorithm is SHA-256 producing a 256-bit hash value.

Claim 4:

The method of claim 1, wherein the digital signature algorithm is RSA with 2048-bit key length.

Claim 5:

The method of claim 1, wherein AI quality validation uses Claude Vision API to analyze photo clarity, lighting, framing, and content matching.

Claim 6:

The method of claim 1, wherein evidence is stored using Write-Once-Read-Many (WORM) storage preventing modification or deletion.

Claim 7:

The method of claim 1, wherein evidence can be captured offline and automatically synchronized when network connectivity is restored.

Claim 8:

The system of claim 2, further comprising multi-region replication with automatic failover for disaster recovery.

Claim 9:

The method of claim 1, wherein the authentication certificate includes a complete chain of custody summary with all access events and actors.

Claim 10:

The method of claim 1, wherein the evidence chain verification algorithm detects tampering in $O(n)$ time complexity for n evidence items.

DRAWINGS (Figures)

Figure 1: System architecture showing hash generator, digital signer, chain linker, AI validator, custody logger, and certificate generator

Figure 2: Multi-component hash generation process showing 7 input components

Figure 3: Evidence chain structure showing blockchain-style linking without blockchain

Figure 4: Custody log chain showing HMAC signatures and chain linking

Figure 5: FRE 902(13) authentication certificate template

Figure 6: Offline capture and sync workflow

DETAILED EXAMPLES

Example 1: NFL Game Day Evidence Capture

Scenario: GDA captures photo of FTR cart positioning for compliance verification

Evidence Capture:

Photo taken: 2025-09-08 10:23:47 UTC

GPS: 33.7553, -84.4006, 315.2m

Operator: GDA-1847

Device: iPhone-12345-iOS-17.1

Task: C2P-HS-001 (FTR Cart Positioning Verification)

Type: Photo

Hash Generation:

```
Content hash: sha256(photo_bytes) = 'e4d5f6g7...'
```

```
Metadata: {  
  'timestamp': '2025-09-08T10:23:47.234Z',  
  'gps': '33.755300,-84.400600,315.2',  
  'operator': 'GDA-1847',  
  'device': 'iPhone-12345-iOS-17.1',  
  'task': 'C2P-HS-001',  
  'type': 'photo'  
}
```

```
Final hash: sha256(content_hash + metadata) =  
'a7b8c9d0e1f2g3h4i5j6...'
```

Digital Signature:

```
Signature = RSA_sign(hash, GDA-1847_private_key)  
           = 'mVq8jK3pL9...' (base64)
```

Verification:

```
RSA_verify(signature, hash, GDA-1847_public_key) = TRUE
```

AI Quality Validation:

Claude Vision Analysis:

```
{  
  "quality_score": 0.92,  
  "passed": true,  
  "validation_notes": "Cart clearly visible, positioned correctly  
per guidelines. Labels readable. Lighting excellent.",  
  "content_match": true  
}
```

Chain Linking:

Evidence entry:

```
{
```

```
"evidence_id": "EVID-2025-09-08-1847-0001",
"evidence_hash": "a7b8c9...",
"previous_hash": "000000..." (first evidence for this operator),
"operator_signature": "mVq8jK3pL9...",
"chain_position": 1
}
```

Custody Log:

```
Event 1: CREATED
  Actor: GDA-1847
  Timestamp: 2025-09-08T10:23:47.234Z
  HMAC: "x1y2z3..."
```

```
Event 2: ACCESSED
  Actor: SUPERVISOR-0042
  Timestamp: 2025-09-08T10:45:12.891Z
  HMAC: "w8v7u6..."
  Previous log hash: "x1y2z3..."
```

Legal Admissibility:

```
FRE 902(13) Certificate generated
Authenticates evidence for court proceedings
No expert witness required for authentication
```

Example 2: Litigation Scenario

Context: Equipment failure lawsuit, evidence from 6 months ago required

Evidence Retrieval:

```
Evidence ID: EVID-2025-09-08-1847-0001
Request: Plaintiff's attorney via subpoena
```

Integrity Verification:

1. Retrieve evidence from WORM storage
2. Recalculate hash from evidence + metadata
Current hash: a7b8c9d0e1f2g3h4i5j6...
Original hash: a7b8c9d0e1f2g3h4i5j6...
Match: TRUE ✓
3. Verify digital signature
Signature: mVq8jK3pL9...
Public key: GDA-1847_public_key
Verification: TRUE ✓
4. Verify evidence chain
Previous hash matches: TRUE ✓
Chain integrity: INTACT ✓
5. Verify custody log chain
All HMAC signatures valid: TRUE ✓
Chain linking intact: TRUE ✓
Log integrity: INTACT ✓

Authentication Certificate:

Generated: FRE 902(13) certificate
Includes: Complete chain of custody
Certified by: System custodian
Status: SELF-AUTHENTICATING (no expert witness needed)

Court Admissibility:

Defense attorney: "How do we know this photo hasn't been altered?"
Plaintiff attorney: "Certificate of authenticity under FRE 902(13).
Cryptographic hash proves no modification.
Digital signature proves it was captured by
GDA-1847.
Complete chain of custody maintained."

Judge: "Evidence admitted."

ADVANTAGES OVER PRIOR ART

Technical Advantages

1. **Multi-Component Hashing:**
2. Prior art: Hash file content only
3. This invention: Hash content + 7 metadata components
4. Benefit: Impossible to substitute evidence from different time/place/operator
5. **Blockchain-Style Without Blockchain:**
6. Prior art: Use blockchain (expensive, slow)
7. This invention: Chain-linking without blockchain
8. Benefit: Instant finality, no transaction costs, works offline
9. **AI Quality Validation:**
10. Prior art: Manual quality review
11. This invention: Automated AI validation
12. Benefit: Immediate feedback, reduced court rejections
13. **Offline Capability:**
14. Prior art: Requires connectivity
15. This invention: Full offline capture + auto-sync

- 16. Benefit: Works in stadiums, remote locations, emergency scenarios
- 17. **WORM Storage:**
- 18. Prior art: Modifiable storage
- 19. This invention: Write-once, immutable storage
- 20. Benefit: Mathematically impossible to alter evidence

Legal Advantages

- 1. **FRE 902(13) Compliance:**
- 2. Prior art: Requires expert witness testimony
- 3. This invention: Self-authenticating certificates
- 4. Benefit: Reduced litigation costs, faster proceedings
- 5. **Non-Repudiation:**
- 6. Prior art: Operator can deny capturing evidence
- 7. This invention: Digital signature proves identity
- 8. Benefit: Stronger legal standing
- 9. **Chain of Custody:**
- 10. Prior art: Manual logs (modifiable)
- 11. This invention: HMAC-signed immutable logs
- 12. Benefit: Tamper-proof audit trail

INDUSTRIAL APPLICABILITY

This invention has broad applicability across industries requiring evidence authentication:

Law Enforcement

- Body camera footage authentication
- Crime scene photography
- Evidence collection and preservation
- Internal affairs investigations

Sports & Entertainment

- Game day compliance verification
- Official replay evidence
- Venue safety documentation
- Broadcast rights enforcement

Healthcare

- Medical imaging authentication
- Surgical procedure documentation
- Clinical trial evidence

- HIPAA-compliant audit trails

Insurance

- Accident scene documentation
- Claims evidence authentication
- Fraud investigation
- Damage assessment verification

Manufacturing

- Quality control documentation
- Safety inspection evidence
- Regulatory compliance proof
- Supply chain verification

Government

- Election integrity verification
- Regulatory enforcement evidence
- Public safety documentation
- Infrastructure inspection records

CLAIMS SUMMARY

This provisional patent application claims:

1. Multi-component cryptographic hash generation for evidence authentication
2. RSA digital signatures for non-repudiation and identity proof
3. Blockchain-style evidence chaining without blockchain infrastructure
4. AI-powered quality validation using computer vision
5. Immutable HMAC-signed chain-linked custody logging
6. FRE 902(13) compliant authentication certificate generation

The invention provides significant advantages over prior art in legal admissibility, tamper-detection, offline capability, and cost efficiency.

END OF PROVISIONAL PATENT APPLICATION

Applicant: NOVATE LABS LLC

Inventor: Tye Hayes

Filing Date: December 27, 2025

Attorney Docket: NOVATE-2025-002-PROV

Note: This provisional application establishes priority date for the evidence chain authentication invention. A non-provisional utility patent application must be filed within 12 months to maintain priority.